01 한눈에 보기

매장 분석 화면의 안내 직원

조회 버튼을 누르면 이 코드가 달과 매장 번호를 챙겨 계산을 부탁하고, 받은 결과를 화면으로 보냅니다.

1. 요청 받기

브라우저가 2026년 10월을 보여 달라고 합니다.

2. 계산 부탁하기

서비스가 테스트 매장 900101의 10월 숫자를 계산합니다.

3. 화면에 보내기

결과를 Model 상자에 담아 HTML이 꺼내 쓰게 합니다.

조회 버튼을 누른 뒤에 보이는 예

```
/analytics/dashboard?month=2026-10
```

가게 번호는 서버 설정에서 가져옵니다. 첨부 예시의 10월 결과는 매출 281,000원, 방문 12건, 방문 고객 6명입니다.

이름 옆에 붙여 읽는 코드 용어

클래스 / 객체	클래스는 설계도입니다. 객체는 그 설계도로 만든 실제 도구입니다. Spring이 DashboardViewController 객체를 만듭니다.
매개변수 / 인수	매개변수는 값을 받는 칸이고, 인수는 호출할 때 그 칸에 보내는 값입니다. 매개변수는 객체를 가리킬 수도 있고 숫자를 담을 수도 있습니다.
필드 / 지역변수	필드는 객체가 보관하는 칸입니다. 지역변수는 메서드나 블록 안에서 잠깐 쓰는 칸입니다.
메서드 / 반환값	메서드는 맡은 일을 하는 코드입니다. 반환값은 그 일을 끝낸 뒤 돌려주는 결과입니다.
애너테이션 / 주석	@로 시작하는 애너테이션은 Spring이 읽는 이름표입니다. Javadoc 주석은 사람이 읽는 설명입니다.

02 앱을 켤 때

안내 직원이 일할 준비를 해요

Spring은 웹 요청을 처리하는 도구입니다. 앱을 켤 때 이 직원에게 계산 도구와 테스트 매장 번호를 건네줍니다.

코드 묶음과 가져오는 도구

package / import	package는 파일이 속한 분석 코드 묶음입니다. import는 다른 묶음의 이름을 짧게 쓰도록 가져옵니다.
Controller / Profile	웹 요청 담당을 등록하고, dev 개발 모드에서만 이 화면을 준비합니다.
Value / Model	Value는 실행 설정을 읽습니다. Model은 화면에 보낼 값을 담는 상자의 사용 방법을 정한 인터페이스입니다.
GetMapping / RequestParam	조회 주소와 실행할 메서드를 연결하고 주소에서 month 값을 꺼냅니다.
DateTimeFormat / YearMonth / ZoneId	날짜 글자의 모양을 읽고, 연도·월을 담고, 서울 시간대를 고릅니다.

클래스와 필드

```
@Controller
@Profile("dev")
public class DashboardViewController {
    private final AnalyticsService analyticsService;
    private final long previewStoreId;
```

analyticsService는 계산 도구 객체를 가리키는 필드입니다. previewStoreId는 매장 번호를 담는 long 정수 필드입니다. private은 클래스 안에서 직접 쓰고, final은 한 번 넣은 값을 다시 바꾸지 못하게 합니다.

생성자에서 받은 값을 보관해요

```
public DashboardViewController(AnalyticsService analyticsService,
    @Value("${app.analytics.preview-store-id:0}") long previewStoreId) {
    this.analyticsService = analyticsService;
    this.previewStoreId = previewStoreId;
}
```

괄호 안의 두 이름은 매개변수입니다. 오른쪽의 받은 값을 왼쪽 this. 필드에 저장합니다. 설정이 없으면 :0에 따라 매장 번호 0을 받습니다. 생성자는 클래스와 이름이 같고 반환 타입을 쓰지 않습니다.

Javadoc의 작성자·작성일·변경노트는 코드 이력을 설명합니다. @param은 입력, @return은 결과 설명입니다.

03 조회할 때

요청이 화면이 되는 네 단계

사용자가 고르는 것은 조회 월입니다. 개발용 매장 번호는 서버 실행 설정에서 읽습니다.

1. 조회 주소를 받고 월을 읽어요

```
@GetMapping("/analytics/dashboard")
public String dashboard(
    @RequestParam(name = "month", required = false)
    @DateTimeFormat(pattern = "uuuu-MM") YearMonth month, Model model) {
```

month는 주소에서 받은 월 객체를 가리키는 매개변수입니다. uuuu-MM은 2026-10처럼 읽는 규칙입니다. required = false라서 생략할 수 있습니다. model은 Spring이 준비한 화면용 상자를 받는 매개변수입니다.

2. 달이 빠졌으면 서울 기준 이번 달을 써요

```
YearMonth selectedMonth = month == null
    ? YearMonth.now(ZoneId.of("Asia/Seoul")) : month;
model.addAttribute("selectedMonth", selectedMonth.toString());
```

조건 ? A : B는 조건이 맞으면 A, 아니면 B를 고릅니다. month가 null이면 서울 기준 이번 달을, 값이 있으면 받은 달을 씁니다. 지역변수 selectedMonth를 글자로 바꿔 상자에 담으면 월 입력 칸에 다시 표시할 수 있습니다.

3. 매장 번호가 있으면 계산을 부탁해요

```
if (previewStoreId > 0) {
    var summary = analyticsService.summarize(previewStoreId, selectedMonth);
    model.addAttribute("summary", summary);
    model.addAttribute("repeatRate", summary.repeatRatePercent());
    model.addAttribute("previewStoreId", previewStoreId);
}
```

번호가 0이면 DB 집계를 건너뜁니다. 번호가 있으면 매장 번호와 달을 인수로 보냅니다. var summary는 계산 결과 객체를 가리키는 지역변수입니다. Model에 집계 결과, 재방문율, 매장 번호를 각각 이름표를 붙여 담습니다.

4. 보여줄 화면을 골라요

```
return "analytics/dashboard";
```

반환값은 화면 이름을 담은 String입니다. Spring이 해당 HTML을 찾으면 Thymeleaf가 Model 속 숫자를 화면에 채웁니다. 이 경로는 개발용이며 점주가 매장 주인인지 검사하는 단계는 아직 없습니다.